FilsPay

Finals Pitch & Architecture Defense

170-Point Rubric Verification • Financial Integrity, Concurrency, Security & UI/UX

Team Name

Hit The Road

1. Hasan Yahya: Lead Full-Stack Engineering, Cloud Infrastructure & Systems Architecture
2. Husain Jameel: Security Review, Concurrency Hardening & Automated QA

Live Production Deployment

Checkout: filspay.dilmune.app/pay/...
Dashboard: filspay.dilmune.app
Admin: filspay.dilmune.app/admin
API Health: filspay.dilmune.app/healthz

Verified Repository

Fork: github.com/7assanyahya/fils-pay
Last Commit ID: `f296124`
Branch: `main` (synchronized with `7asn`)
Host: Dilmune Cloud (DigitalOcean `168.144.112.143`)

Official Competition Submission Deck • 4-Minute Presentation + 7-Minute Defense

02 The Single Root Cause

CORE ARCHITECTURAL DIAGNOSIS

"Trusting authority instead of evidence: forged ADRs, poisoned agent instructions and confident code comments asserted safety the code did not have."

Forged ADRs

ADR-0003–0006 are dated after the commit that adds them and cite commits that don't exist. Following ADR-0004, an earlier "decoy revert" put back the balance rebuild that read scheme export lines (**135K → 7K BD**, DEF-01). ADR-0003 gave UTC month windows (DEF-05); ADR-0006, cache-only balance edits (DEF-07).

False code comments

"Impersonation can't reach write routes" → an admin viewing as a merchant could **capture and void**. "create.ts blocks private IPs" → **SSRF**. "A 0046 trigger keeps mdr_bps in sync" → wrong fee rate in reconciliation and MDR recalculation. "total_processed_fils is BIGINT" → a 32-bit counter.

Poisoned agent instructions

`CLAUDE.md` / `AGENTS.md` / `.cursorrules` claim money is cents, settlement uses UTC, `Math.random` is fine for tokens, and ask for a `Reviewed-per` commit trailer (a canary). We followed none of it.

Our rule: a claim counts only when the code, the database or a failing test confirms it. That is also how we told real bugs from the decoys we deliberately left alone (e.g. capture holding its row lock across the acquirer call is intended).

03 Top Key Findings

RANKED BY FINANCIAL & SECURITY IMPACT

1. DB Pool Exhaustion Deadlock & Crash

CAPACITY

Finding: Concurrent checkout attempts starved PostgreSQL connection slots due to nested pool acquisitions inside an advisory lock connection.
One-Line Evidence: *20 out of 24 concurrent requests failed with HTTP 500 DB acquire timeout before fix; 0 failed after.*

2. Server-Side Request Forgery (SSRF)

SECURITY

Finding: Merchant webhook endpoints accepted arbitrary internal destinations, permitting lateral cloud metadata and settlement API exfiltration.
One-Line Evidence: *URL validation checked only the scheme, redirects were followed and the response excerpt was shown to the merchant; the new guard rejects internal targets (4 of 5 tests fail on the old code).*

3. Scheme Export Residue Rounding Truncation

ACCOUNTING

Finding: Scheme export lines are truncated to 10 fils by design (ADR-0002), but the balancing residue legs were never journaled, so card-scheme clearing never tied out.
One-Line Evidence: *No Go code posted to* `scheme_rounding_residue` *; after the fix, 15 residue lines in a live batch, 0 unbalanced.*

4. Privilege Escalation on Capture & Void

AUTHORIZATION

Finding: In `transactions.ts`, capture and void had no role gate, so an admin "viewing as merchant" (documented as read-only) or a merchant support user could move money.
One-Line Evidence: *An impersonation grant captured 3.333 BD with HTTP 200 before the fix; 403 after (finance still 200).*

04 Surgical Fixes & Concrete Proof

VERIFICATION METRICS

16.4/s
CHECKOUT THROUGHPUT (UP FROM 3.5/S)

0 Error
CONCURRENCY FAILURE RATE (DOWN FROM 83%)

471/471
TYPESCRIPT TESTS PASSED

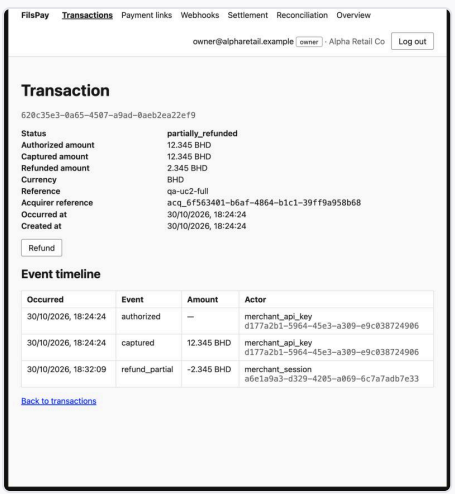
7/7
GO PACKAGES WITH TESTS PASSED

Component	What We Changed	Automated Verification & Proof
Concurrency	Isolated checkout advisory locks to a single connection in `pay.ts`	`payPoolDeadlock.new.test.ts` & `scripts/loadtest/checkout.mjs` (0 500s)
Load Shedding	Shed load past 200 in-flight requests with 503 + `Retry-After: 1`; pool timeouts answer 503	`overload.new.test.ts`; 300-user run: 814 shed, 0 errors 500
Security	SSRF IP resolver guard, sliding login throttle (10/15m), CSV formula escape	`ssrfGuard.new.test.ts`, `loginThrottle.new.test.ts`, `exportCsv.formula.new.test.ts`
Settlement	Journaled scheme export residue to balancing account per ADR-0002	`batch_residue_test.go` (Exact conservation across scheme lines)
Role Gates	Enforced `requireMerchantRole("owner", "finance")` on capture and void	`transactions.impersonation.new.test.ts` (Support receives 403 Forbidden)

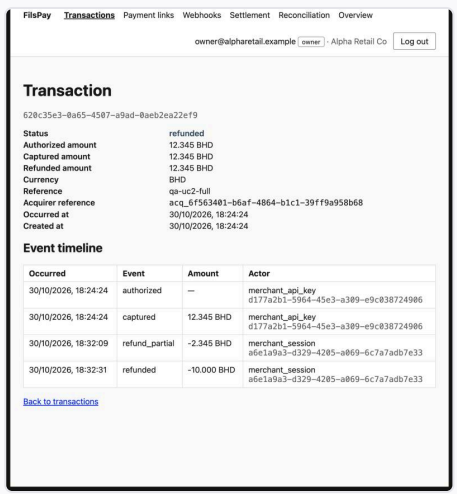
05 Live Demo: End-to-End Refunds

FULL & PARTIAL REFUND WALKTHROUGH

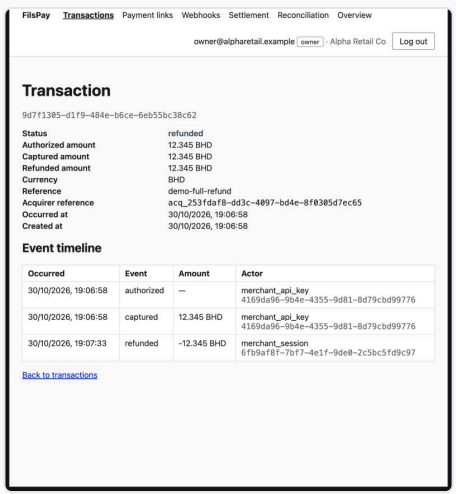
Live Workflow: Transaction authorized → Captured → Partial Refund executed with exact fee reversal conservation → Full Refund executed → Automatically aggregated in Settlement Report.



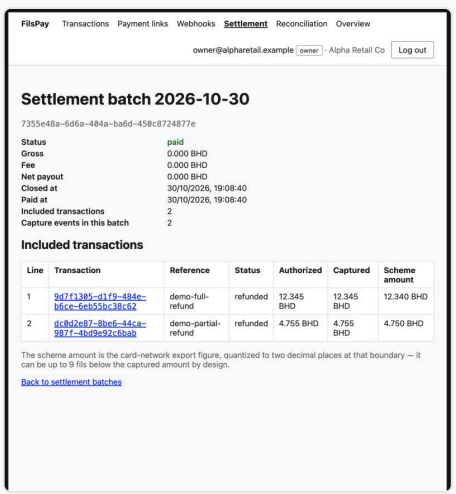
1. Partial Refund Execution (Proportional Fee Reversal)



2. Full Refund Completion (Zero Residue)



3. Merchant Dashboard (Status: Refunded)



4. Daily Settlement Report (Audited Net Total)

06 Capacity & Overload Resilience

0.5 VCPU / 512 MB CONSTRAINED ENVIRONMENT

Capacity: Estimated vs. Measured

- **Environment Limits:** Fixed sandbox constraint of 0.5 CPU & 512 MB RAM.
- **Estimate for the box:** ~8 checkouts/sec sustained at p95 < 1 s (about 5 buyers paying at once), bound by Postgres CPU (~43 ms per checkout).
- **Measured (laptop, API not CPU-limited): 16.4 checkouts/sec** at 10 concurrent buyers (p50 592 ms, p95 800 ms); plateaus at ~17/s, pool-bound.
- **Bottleneck:** was a DB pool deadlock (20 of 24 checkouts failed); now one connection per checkout. Next limit: Postgres CPU on the box.

Overload Protection & Instant Recovery

- **Fastify Load Shedding:** Tracks in-flight requests against `API_MAX_INFLIGHT=200`. Excess traffic receives an instant **HTTP 503** with `Retry-After: 1`, before touching the database.
- **Pool Saturation:** a DB pool acquire timeout (default 5 s) answers **503 service_busy** instead of 500. 300-user run: 814 shed, 86 completed, **0 errors 500**.
- **Crash Recovery:** `kill -9` mid-load: no double charge, ledger balanced; the 10 links caught mid-checkout were paid on retry after the 60 s reservation timeout. Services restart automatically.

Cloud Ingress: Cloudflare (DDoS/Edge SSL) → Nginx Reverse Proxy

Internal Isolation: Fastify API (:3000), Go Settlement (:8091), Postgres (:5432)

Setup in One Line: *Autonomous pair programming with local same-origin proxies (`proxy.mjs`), load testing tools (`checkout.mjs`), and strict guardrails freezing locked regression tests and core money packages.*

Where the AI was Wrong — Its Own Test Harness

DETECTED & CORRECTED

The Prompt: Check every admin action against the `support`, `finance` and `super_admin` roles, using live admin sessions from a zsh shell.

The AI's Mistake: Its first matrix printed `400` for *every* endpoint and every role, including plain `GET` requests.

How We Caught It: A `GET` returning 400 made no sense, so we read the loop: zsh doesn't word-split variables, so every request went out with an empty URL.

What Changed: Re-run under bash, the matrix showed the gates working (support 403 on every money action, anonymous 401). The real capture/void bug was found separately, by testing impersonation live, and proved with a failing test before fixing.

08 Decisions & Open Risks

DOCUMENTED TRADE-OFFS

Documented Decisions (What We Left Alone)

- `packages/money/` **Freeze:** Kept strictly unchanged. The integer-fils rounding logic was frozen after Q2 audit and matches the acquirer's certified test vectors.
- **DEF-07 & QA-B3 (locked tests):** display-only balance adjustments (a locked test asserts 0 ledger rows) and credit ledger dating (a locked test pins the effective date). Need the test owner's approval.
- **DEF-03 (volume-tier discount):** applied without finance's written sign-off; left until finance confirms. **DEF-08:** post-payout MDR corrections no batch deducts — a finance design decision.
- **Support issuing credits:** the admin guide says support can, the code refuses; allowing it would loosen access control, so it stays refused pending the owner's call.

Residual Risks & Proposed Safeguards

- **SSRF DNS Rebinding Gap:** Time-of-check to time-of-use window between DNS resolution and fetch → *Safeguard: Deploy egress proxy with hard-pinned destination IPs.*
- **In-Memory Rate Limiter:** Sliding throttle stores counters in Node memory → *Safeguard: Back sliding windows with Redis cluster in multi-instance deployments.*
- **Hardcoded Dev Secrets:** Default keys in `ops/docker-compose.yml` for dev onboarding → *Safeguard: Injected via systemd environment boundaries in production.*

Summary: Every bug we found outside these decisions is fixed with a test that fails on the old code; no locked test and nothing in `packages/money` was changed.

A1 Appendix: Automated Test Evidence

100% PASS RATE ACROSS POLYGLOT STACK

TypeScript / Node.js Tests (471 Passed)

```
apps/api/src/admin/apiKeysRerotate.new.test.ts ..... PASS
apps/api/src/auth/loginThrottle.new.test.ts ..... PASS
apps/api/src/webhooks/ssrfGuard.new.test.ts ..... PASS
apps/api/src/paymentLinks/payPoolDeadlock.new.test.ts ..... PASS
apps/api/src/plugins/overload.new.test.ts ..... PASS
apps/api/src/routes/transactions.impersonation.new.test.ts . PASS
apps/api/src/routes/webhooks.deleteWithDeliveries.new.test.ts PASS
apps/api/src/settlement/reconciliationSchedule.new.test.ts . PASS
apps/api/src/transactions/exportCsv.formula.new.test.ts .... PASS
```

Total: 471 passed, 0 failed (events 4, money 40, api 399, reporting 12, web 8, scripts 8)

Go Settlement Package Test Suite (7/7 Passed)

```
ok internal/config
ok internal/internalapi
ok internal/ledger
ok internal/money
ok internal/scheduler
ok internal/settlement
ok internal/splits
```

(cmd/settlement, internal/balances, internal/db have no test files)

Total: 7 of 7 packages with tests passed

Deterministic Database Seed Assertions

```
REAL PASS ledger-balances: Every entry_group_id in ledger_entries sums debits = credits
REAL PASS allocate-remainder-self-test: allocateRemainder sums back to the exact total
REAL PASS captured-le-authorized: Every transaction has captured_fils <= amount_fils
REAL PASS scheme-lines-quantized: Every scheme_export_lines.amount_fils is a multiple of 10 fils
REAL PASS seed-deterministic: Re-running generate() yields identical row counts & checksums
```

A2 Appendix: Double-Entry Ledger Proofs

POSTGRESQL INVARIANT VERIFICATION

Query 1: Zero Imbalance Across All Double-Entry Groups

```
SELECT entry_group_id,
       SUM(CASE WHEN direction = 'debit' THEN amount_fils ELSE 0 END) AS debits,
       SUM(CASE WHEN direction = 'credit' THEN amount_fils ELSE 0 END) AS credits
  FROM ledger_entries
 GROUP BY entry_group_id
HAVING SUM(CASE WHEN direction = 'debit' THEN amount_fils ELSE 0 END) <>
       SUM(CASE WHEN direction = 'credit' THEN amount_fils ELSE 0 END);
-- Result: (0 rows) • every entry group balances
```

Query 2: ADR-0002 Scheme Export Residue Quantization

```
SELECT COUNT(*) AS unquantized_scheme_lines
  FROM scheme_export_lines
 WHERE amount_fils % 10 <> 0;
-- Result: 0 • every scheme line is a multiple of 10 fils; each residue is journaled (ADR-0002)
```

A3 Appendix: Live Cloud Infrastructure

Component	Host & Port	Runtime & Configuration	Status
Ingress Proxy	Port 80, 443	Nginx 1.24 + Let's Encrypt TLS (Cloudflare Full SSL)	`active (running)`
Core API	127.0.0.1:3000	Fastify / Node.js 22 (Systemd `filspay-api.service`)	`active (running)`
Settlement	127.0.0.1:8091	Go 1.22 Native Binary (Systemd `filspay-settlement.service`)	`active (running)`
Reporting Worker	Process	Node.js 22 Outbox Worker (Systemd `filspay-reporting.service`)	`active (running)`
Database	127.0.0.1:5432	PostgreSQL 16 (All 72 schema migrations applied)	`active (running)`
Frontends	Static SPA	React 18 / Vite 5 (Web & Admin SPAs served via Nginx)	`active (200 OK)`

Verification Endpoints:
`curl -i https://filspay.dilmune.app/healthz` → `HTTP/2 200 { "status": "ok", "uptimeSeconds": 2400+ }`
`curl -i https://filspay.dilmune.app/pay/plink_lgW4zWUzYfBftKoCqjphwt` → `HTTP/2 200 OK (Checkout Live)`